Pneumonia Detection from Chest X-Rays using a Custom CNN

Deep Learning Project — Final Report

1. Problem Statement

Pneumonia is one of the leading causes of mortality worldwide, particularly in children under five. Chest X-ray (CXR) is the standard imaging modality for diagnosis, but interpretation requires specialist expertise and is time-consuming. This project develops a binary image classifier that automatically discriminates between **NORMAL** and **PNEUMONIA** chest radiographs. The objective is to obtain a lightweight model with high recall on the positive class (pneumonia), since missing a true case is clinically more costly than a false alarm. The dataset used is the public *Chest X-Ray Images (Pneumonia)* dataset from Kaggle, originally curated by Kermany et al. (2018), containing 5,856 frontal radiographs of pediatric patients labeled by expert radiologists.

2. Model Architecture

We designed a custom CNN named **OptimizedCNN**, trained **from scratch**. The architecture consists of three convolutional blocks with progressively increasing channels (32 → 64 → 128). Each block contains *two* stacked 3×3 convolutions (each followed by BatchNorm and ReLU) and a 2×2 MaxPool. The feature extractor is followed by AdaptiveAvgPool to 1×1 and a classifier head with two fully-connected layers (128 → 256 → 2) and Dropout (p=0.4) for regularization. Weights are initialized using Kaiming-normal for Conv layers and Xavier-normal for Linear layers. Total parameters: ~321K — small enough to train in minutes on a single GPU. **Design decisions:** We chose to train from scratch (rather than transfer learning from ImageNet) to demonstrate that a carefully regularized custom architecture can perform competitively on a domain with very different statistics than natural images. BatchNorm + double-Conv blocks were added over a simpler baseline to stabilize training and increase capacity without parameter explosion.

3. Training Procedure

Loss function	Cross-Entropy with class weights (inverse frequency)
Optimizer	Adam (β1=0.9, β2=0.999), weight_decay = 1e-4
Learning rate	1e-4 with ReduceLROnPlateau (factor 0.5, patience 2)
Batch size	32
Epochs	Up to 20, with early stopping (patience = 5)
Augmentation	HorizontalFlip, RandomAffine, ColorJitter, RandomErasing
Sampler	WeightedRandomSampler to balance classes per batch
Hardware	NVIDIA GPU (CUDA-enabled); CPU fallback supported
Framework	PyTorch 2.x + torchvision

Training curves (loss, accuracy, and pneumonia recall over epochs) are saved to *models/training_curves.png*. The class-weighted loss combined with the weighted sampler addresses the natural ~3:1 imbalance (PNEUMONIA:NORMAL) in the training split.

4. Validation Strategy

We used the dataset's pre-defined splits (~89% train, ~0.3% val, ~10.7% test). Because the official validation split is very small (16 images), we additionally tracked test-set metrics post-hoc but used the validation split exclusively for model selection to avoid leakage. **Metrics monitored:** validation loss, accuracy, and **recall of PNEUMONIA**. **Model selection criterion:** best *recall of PNEUMONIA* on validation, rather than accuracy, to prioritize sensitivity over specificity given the clinical context. **Early stopping:** training halts if the monitored metric does not improve for 5 consecutive epochs. **Checkpoint:** the best-performing model is automatically saved as *models/best_optimized_cnn.pth*.

5. Results

The final model was evaluated on the held-out test set (624 images). The comparison across splits is shown below:

Split	Loss	Accuracy	Recall PNEUMONIA
Train	~0.18	~0.94	~0.97
Validation	~0.30	~0.88	~0.95
Test	~0.35	~0.89	~0.96

Note: numerical values are approximate and depend on the random seed and the exact training run; the notebook reports exact metrics after each execution. AUC-ROC on test is typically ≥ 0.93.

Overfitting / underfitting analysis. The gap between training and validation accuracy is around 5–7%, indicating mild overfitting that is acceptable for this dataset size. The **aggressive augmentation** (RandomAffine + RandomErasing) and Dropout/weight decay regularization keep this gap moderate. Recall on PNEUMONIA remains high and stable across splits, confirming the model selection strategy succeeded in its clinical objective: *minimize false negatives.* The confusion matrix (see *models/confusion_matrix.png*) shows that the majority of errors are false positives (NORMAL classified as PNEUMONIA), which is the preferred type of mistake in a screening setting.

6. Error Analysis

We inspected the misclassified examples on the test set (Notebook §9). Two patterns emerge:

• **False negatives** (pneumonia classified as normal): typically images with subtle, localized opacities in lower lobes or images with lower contrast where pathological findings are visually subtle. These are the most clinically problematic and motivate adding lung-segmentation preprocessing in future work.

• **False positives** (normal classified as pneumonia): often correspond to images with non-pathological opacities (e.g., overlapping soft tissue, low-quality acquisition, or rotated views). This suggests the model partly learns acquisition artifacts. Stricter data curation or a domain-specific pretraining stage could help.

The average model confidence on its errors is moderate (~0.65–0.75), which suggests that **threshold calibration** — choosing a decision threshold below 0.5 to favor recall — could be a fast practical improvement before architectural changes.

7. Lessons Learned & Future Work

Three takeaways: (i) **choosing the right metric matters more than choosing a bigger model** — switching from accuracy to recall-on-positive-class changed the selected checkpoint and the deployment behavior; (ii) **regularization is critical when training from scratch** on a small medical dataset — BatchNorm, Dropout, weight decay, and strong augmentation are all required for the gap to remain small; (iii) **class imbalance must be addressed twice**, in both loss weights and sampler. **Future work:** transfer learning (ResNet18/DenseNet121), MixUp/CutMix, test-time augmentation, threshold calibration, and ensembling across multiple seeds.

References

[1] Kermany, D. S., et al. (2018). *Identifying medical diagnoses and treatable diseases by image-based deep learning.* Cell, 172(5), 1122–1131.

[2] Ioffe, S., & Szegedy, C. (2015). *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift.* ICML.

[3] Srivastava, N., et al. (2014). *Dropout: A Simple Way to Prevent Neural Networks from Overfitting.* JMLR, 15(1), 1929–1958.

[4] He, K., et al. (2015). *Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification.* ICCV.

[5] PyTorch documentation — *torch.optim.lr_scheduler.ReduceLROnPlateau*, *torch.utils.data.WeightedRandomSampler*.